

Asymptotic

Additional correctness findings,
Wolfram Language comparison,
and a contract-preserving repair plan

Technical audit prepared for Vladimir Reshetnikov

9 September 2026

Four native-reproduced findings

Proof context Parameter dependence Regularity Realness

Repository

<https://github.com/VladimirReshetnikov/Asymptotic>

Pinned revision

921387e5ba1239bfda96e63e64e89bf63d9c41e6

Observed audit kernel

Wolfram Language 15.0.0 for Linux x86 (64-bit), build dated May 6, 2026.

Evidence boundary. The four principal findings were reproduced in a native Wolfram kernel. The mathematical witnesses are derived explicitly. Fifteen independent Python checks passed. The full upstream native suite was not run, and proposed patches were not successfully integration-tested in the native kernel. Historical validation, new observations, and unexecuted regression specifications are distinguished throughout.

Abstract

This audit examines the current Asymptotic repository against its existing nine-review register, emphasizing additional evidence rather than repeating the backlog. Four successful native evaluations expose distinct failures at contract boundaries. An ambient assumption can turn an excluded source interval into a positive inverse certificate. Composition can capture a fixed parameter as its new expansion variable and return $1 + \mathcal{O}(a^2)$ for a function identically equal to $1/2$. A real opaque function with a correctly supplied first derivative can receive a false magnitude remainder because formal Taylor generation imports an unstated analyticity assumption. Finally, direct expansion and an observable route admit a provably nonreal symbolic coefficient that scalar addition and inverse construction correctly reject.

These findings do not show that the package’s specialized coefficient formulas are generally incorrect. They show that correct local algebra is insufficient when proof context, parameter quantifiers, regularity, or codomain evidence changes between operations. The article gives source locations, observed outputs, exact witnesses, narrowly scoped repairs, acceptance tests, and performance-conscious implementation proposals. It also compares the package with the actual breadth of Wolfram’s native asymptotic facilities, rather than treating those facilities as merely Taylor-series tools. A source-pinned certificate-patch generator, native reproducers, desired-contract tests, independent checks, and evidence manifests accompany the article.

Contents

1	Executive assessment	2
2	Scope, novelty, and evidence	3
3	The mathematical and software layers being compared	4
4	Comparison with built-in Wolfram Language	5
5	N01: ambient assumptions can create a false branch certificate	7
6	N02: composition promotes a fixed-parameter bound to a false uniform bound	9
7	N03: a truthful finite jet does not establish an analytic remainder	11
8	N04: codomain admission is inconsistent across public routes	13
9	A repair architecture focused on the demonstrated defects	15
10	Performance and UX implications specific to these findings	16
11	Acceptance plan and reproducibility	17
A	Source coverage and limits	19
B	Exact witnesses and useful variants	19
C	Artifact manifest and build instructions	20

1 Executive assessment

1.1 What should change first

The first repair priority is the certificate’s proof context. A positive `Certified` flag should not depend on an unrelated notebook-level assumption about the source variable. The next priority is composition’s treatment of fixed parameters: a successful composition must not silently turn a parameter-dependent bound into a uniform one. The ordinary forward fallback then needs an explicit regularity admission rule, and public real-valued operations need consistent codomain admission.

ID	Priority	Native-reproduced finding	Added evidence
N01	Critical contract	The certificate reports the root 1 as satisfying the retained source domain $x > 2$.	False positive certificate, not just missing assumption metadata.
N02	High	Composition reports $1 + \mathcal{O}(a^2)$ for $a/(a+a) = 1/2$.	Concrete quantifier escape from fixed-parameter asymptotics.
N03	High	A real C^1 opaque function receives an invalid $\mathcal{O}(x^2)$ tail.	Truthful finite derivative data do not establish the claimed remainder.
N04	Medium	Direct expansion and an observable admit $\sqrt{-a}$ under $a > 0$; scalar addition and inverse construction reject it.	Cross-route codomain inconsistency with positive rejection controls.

“Critical” in N01 means a failure of an explicit correctness certificate, not a claim of a security vulnerability or an estimate of deployment impact. N04 has a different character from N01–N03: its displayed expression is algebraically correct as a complex expression. The defect is an inconsistent real-valued API boundary, not an incorrect complex coefficient calculation.

1.2 What remains valuable in the design

The package exposes a useful abstraction beyond a displayed approximation: a `GeneralizedSeries` carries a local coordinate, finite coefficients, a remainder description, assumptions, and, in several families, source equations and evidence for later operations. The ordinary sparse power–log engine is complemented by exact-core inversions, finite flat sectors, Fourier-polynomial logarithmic coefficients, reciprocal-logarithmic calculus, and Gamma/Barnes inverse scales. These are cooperating specialized algebras, not a single universal transseries field.[1, 9, 10, 12, 14, 15, 16, 17]

The inspected special-function machinery also contains substantive safeguards. Exact identities are separated from truncated asymptotic formulas; half-integer modified Bessel identities retain both exponentials when required. Finite flat-sector inversion keeps the zero sector exact instead of disguising an algebraically truncated inverse as an exact core. The inverse-Gamma operation code explicitly prevents source replay from improving uncertainty inherited from an operand. These choices are worth preserving while repairing the newly demonstrated boundaries.[19, 12, 18]

The resulting recommendation is not a rewrite of the coefficient engines. It is a small set of stronger admission and transport rules around them, with explicit proof obligations and regression cases that make the repaired behavior observable.

2 Scope, novelty, and evidence

2.1 Source pin and execution environment

The source identities are recorded separately to avoid conflating a commit, a Git blob, and a distribution checksum.

Repository commit

921387e5ba1239bfda96e63e64e89bf63d9c41e6

Generated standalone Git blob

609eaae41eac2a0f00c9c898b22265375add4306

Canonical certificate-module Git blob

c447755ab58f02aa8a80fa374329e00d6ff9641e

The generated root file instructs maintainers to edit the modular kernel and regenerate with `validation/build_standalone.py`. The supplied patch generator follows that ownership boundary: it targets the canonical certificate module and does not silently edit a distribution file.[6, 7, 11]

Repository files were read through the GitHub connector. A complete checkout was not available in the local artifact-building environment. The native evaluator did successfully download and load the pinned standalone package for the substantive probes. Each successful probe recorded its actual output; larger or repeated calls also encountered intermittent service errors. A connector network failure is not counted as a package exception, an algorithmic timeout, or a failed mathematical test.

The native kernel identified itself as `15.0.0 for Linux x86 (64-bit) (May 6, 2026)`. This is the observed kernel, not a claim about the newest available Wolfram release. The repository advertises a Wolfram Language 15.0-or-later requirement and package version 1.8.0; neither the version label nor earlier validation logs substitute for executing the current cases.[1]

2.2 How this report avoids repeating the existing reviews

The existing review index and consolidated status register were used as the deduplication baseline. Targeted historical material was then inspected, including the beginning of review 4 and the execution/patch account in review 8. This is not a claim that every page of all nine historical articles was reread. The register is the main cross-review index.[2, 3, 4, 5]

New ID	Prior register	Why the present finding adds information
N01	C05	Prior ambient-assumption concerns become a native positive certificate for a root outside the stored source domain, with a specific vulnerable <code>Refine</code> call.
N02	X10, C13	The distinction between fixed and varying parameters is now a public incorrect result, not merely a future uniform-asymptotics feature or a generic composition-domain question.
N03	C06	An explicit opaque function with correct value and derivative defeats the promised magnitude bound. The cause is a formal-to-analytic trust transition, not only a possible missing logarithmic degree.

New ID	Prior register	Why the present finding adds information
N04	C07	Previously a real-coefficient validation concern; now native acceptance and rejection controls identify the inconsistent public routes.

The register’s already-addressed dense-export and nested uncertain-power findings are not reissued here. Nor are its broad performance, provenance, release-testing, integration, multivariate, complex-sector, or general uniformity roadmaps repeated as discoveries. Later proposals are restricted to the additional obligations exposed by N01–N04 and one specific safe-but-unsupported observable case.[3]

2.3 Evidence categories

An *observed result* below is an output returned by the native Wolfram connector on the pinned package. A *mathematical extension* is proved independently but was not necessarily executed through that connector. A *source deduction* identifies a code path or representation property. A *proposal* describes a change not yet established as a native integration success.

The bundle contains ten executed Python mathematical checks and five executed patch-mechanics checks. The latter operate on explicit source-anchor fixtures; they do not validate the full upstream module or apply the patch to a complete checkout. The twelve Wolfram desired-contract tests are specifications supplied for subsequent execution. Some are intentionally expected to fail on the reviewed baseline. Neither those twelve tests as a suite nor the full upstream suite was run here.

3 The mathematical and software layers being compared

3.1 Ordinary jets and exact carriers

Write $u \rightarrow 0^+$ for the package’s positive local coordinate. The ordinary representation is structurally a finite sparse jet

$$J(u) = \sum_{\beta \in B} u^\beta P_\beta(\log u) + \mathcal{O}\left(u^\rho(1 + |\log u|)^d\right), \quad (1)$$

where the ordered source weights are exact real numbers and the coefficients are logarithmic polynomials. Internally, a precision-tracked triple stores the rows and the pair (ρ, d) . Equal weights are merged before the relevant truncation operations. The derived-series layer can retain an exact carrier and offset,

$$b(y) + C(y)(J(w(y))), \quad w(y) > 0, \quad (2)$$

so that the jet’s order can be relative to C rather than an absolute power of y . [9, 10]

These representations make algebraic precision transport practical. For example, if A has leading power α and error power P , while B has leading power β and error power Q , multiplication must account for errors at powers $P + \beta$, $Q + \alpha$, and, when relevant, $P + Q$. A precision representation cannot be inferred merely by multiplying the printed finite parts. This is mathematical reasoning, independent of any particular implementation.

The four findings concern assumptions around such algebra. In N02 the hidden constant in (1) is allowed to depend on a parameter that later varies. In N03 the existence of the stated error class was never established. Correct exponent arithmetic cannot repair either loss of information.

3.2 Specialized families have different coefficient domains

The flat engine keeps exponential sector degree separate from inner algebraic precision. Its operation layer propagates the omitted exponential tail separately from discarded coefficients inside retained sectors. The Fourier engine instead uses finite modes $P_\omega(\log u)e^{i\omega \log u}$ and checks conjugacy to represent real coefficients. Its error envelope uses coefficient magnitudes, not a possibly vanishing oscillatory factor.[12, 13, 14]

The reciprocal-logarithmic operations use a carrier times a Taylor jet in a positive reciprocal logarithm. Gamma and Barnes inverse expansions keep complete polynomials in an inverse logarithm at each power of an exact large inverse core. Their source formulas and declared Poincare remainder contracts differ from a convergent Taylor model. General operations explicitly refuse several unsupported combinations rather than converting every family into an ordinary power-log jet.[15, 16, 17, 18, 10]

This specialization is an architectural strength when the restrictions are explicit. A refusal because an operation requires a larger coefficient algebra is not an incorrect result. The dangerous case is a successful conversion that silently changes the meaning of an existing bound or proof.

4 Comparison with built-in Wolfram Language

4.1 The native baseline is broader than Taylor reversion

Wolfram’s **Series** supports negative and fractional powers, logarithms, expansions at infinity, and more than univariate Taylor-polynomial output. Its parameter and regularity assumptions matter. **Asymptotic** is a separate general asymptotic interface, while **AsymptoticSolve** addresses implicit equations and systems with automatically inferred scales. Comparing only with **InverseSeries** would therefore understate the native baseline.[22, 26, 27]

Task	Built-in baseline	Repository’s relevant specialization
Ordinary local expansions	Series , SeriesData , ordinary series arithmetic.	Sparse exact-real weights, explicit logarithmic error envelopes, persistent source/scale metadata.
Local inverse series	InverseSeries reverses suitable SeriesData , including documented fractional-power examples.	Source-endpoint and real-side selection; inverse observables; retained model and residual/certificate routes.
Implicit asymptotics	AsymptoticSolve supports systems and inferred scales, not only polynomial reversion.	Named specialized inverse engines and inspectable per-family truncation/remainder contracts.
Composition	ComposeSeries composes appropriate series objects and handles available order.	Composition across selected carrier/coordinate representations, plus reciprocal-logarithmic specialization. N02 exposes a missing dependency guard.
Special functions	Series and Asymptotic are broad native backends.	Exact-identity normalization, family recognition, and transport into persistent expansion objects.

Task	Built-in baseline	Repository's relevant specialization
Proof context	Native simplification uses documented assumption semantics.	A certificate must isolate and record the assumptions appropriate to its stronger interval statement. N01 fails at that integration boundary.

The first four rows are grounded in the native reference descriptions and inspected package representation/operation code; the last two are comparisons of responsibilities rather than claims of a universal native limitation.[22, 23, 24, 25, 26, 27, 10, 20, 11]

4.2 Representation is not the same question as computability

`SeriesData[x,x0,coefficients,nmin,nmax,den]` represents its displayed powers using the rational lattice determined by its integer indices and denominator. This is a representation constraint. It does not imply that Wolfram Language cannot manipulate irrational powers in expressions or compute asymptotics involving them through other interfaces.[23]

The package's sparse exact-real support is therefore a genuine representation advantage for its selected algebra, but not proof of an exclusive computational capability. The fair comparison is whether a given task produces the needed coefficients, branch, usable residual, and retained precision contract, and whether later operations preserve that contract. The supplied optional comparison harness records individual outputs without converting a refusal into a claim that the native system can never solve the mathematical problem.

4.3 Orders must be aligned before comparing answers

A native `AsymptoticSolve` order is an approximation-order request and need not equal polynomial degree; its documentation describes error relative to the final scale term. The package, depending on family, uses an exclusive exponent cutoff or an inclusive sector depth. Equal integers in two calls do not automatically request equal information.[27, 12, 14, 16]

For an ordinary comparison such as $f(x) = x + x^2$, one should align the actual retained powers, not just pass the same integer to both interfaces. For a Gamma inverse comparison, align the exact large core and the retained inverse-logarithmic coefficient blocks. For a flat expansion, state separately the number of exponential sectors and any coefficient truncation. This is a comparison protocol, not a newly reported order-API defect.

4.4 Documented defaults become obligations at the wrapper boundary

Two native defaults are directly relevant. `Refine` appends its default assumption context to positional assumptions unless an explicit assumption option replaces it. `Series` defaults to treating unrecognized functions as analytic. Those are documented native behaviors, not themselves Wolfram defects. A wrapper promising a source-domain certificate or a magnitude remainder must account for them.[28, 22]

The key distinction is the strength of the advertised result. A symbolic Taylor expression under a convention about unknown functions, a Poincare approximation under stated hypotheses, and a verified enclosure for a constrained source root are different outputs. The wrapper may rely on native algorithms, but it must not silently promote one kind of evidence into another.

5 N01: ambient assumptions can create a false branch certificate

5.1 A small public reproducer

All examples below use the pinned package loaded in a fresh kernel. The fully qualified forms are retained to avoid dependence on the reader's context path.

```
Clear[x, y];
s = AsymptoticInverse'AsymptoticInverse[
  ConditionalExpression[x, x > 2],
  {x, Infinity}, {y, 2}];

clean = AsymptoticInverse'InverseCertificate[
  s, 1, "Interval" -> {1/2, 3/2},
  "Center" -> 1, "MaxRefinements" -> 0];

ambient = Assuming[x > 2,
  AsymptoticInverse'InverseCertificate[
    s, 1, "Interval" -> {1/2, 3/2},
    "Center" -> 1, "MaxRefinements" -> 0]];
```

The object is constructed *before* entering **Assuming**. It retains the source condition $x > 2$, the expression y , and **Assumptions** -> **True**. The clean certificate correctly returns **Failure["OutsideBranch",...]** and records that the source condition was not proved on the entire closed verification interval.

Under the ambient assumption, the observed output instead contains:

```
<|"Certified" -> True,
  "RootEnclosure" -> {1, 1},
  "CertifiedSourceDomain" -> x > 2,
  "SourceDomainVerified" -> True,
  "CertifiedFunction" -> x,
  "CertifiedTarget" -> 1|>
```

These fields are transcribed from the successful native response; the full surrounding diagnostic material is not necessary to demonstrate the contradiction.

5.2 What is false, precisely

The unrestricted equation $x = 1$ has root 1. The numerical root is not the defect. The specified inverse source domain is $x > 2$, on which the equation has no solution. Moreover, the entire interval $[1/2, 3/2]$ is outside that domain. Thus the displayed root enclosure and the displayed positive source-domain claim cannot both describe the requested constrained inverse.

This is not saved by interpreting the ambient assumption as another ordinary parameter hypothesis. The certificate is quantifying over an explicit verification interval in the source variable. It cannot use the desired source predicate itself as an unrecorded premise to declare that the interval satisfies the predicate. The source variable must be treated as bound by the interval verification, not as a free fixed parameter whose value is already known to exceed two.

5.3 The exact source-level cause

The current **certPositiveCondition** begins with:


```
c = TimeConstrained[
  Quiet[Refine[condition, Element[x, Reals]]],
  1, condition];
Which[c === True, True, c === False, False, ...]
```

The remaining helper has substantial interval logic: Boolean decomposition, exact affine endpoint evaluation, and rational enclosure comparisons. That machinery is bypassed when the first simplification has already reduced the predicate to **True**.^[11]

The independent native primitive check returned

```
Assuming[x > 2, Refine[x > 2, Element[x, Reals]]]
(* True *)

Refine[x > 2, Element[x, Reals], Assumptions -> True]
(* x > 2 *)
```

This matches the official assumption semantics.^[28]

5.4 Narrow repair and its limits

The minimal source candidate changes that call to:

```
Refine[condition, Element[x, Reals], Assumptions -> True]
```

It preserves the intended realness normalization while preventing this specific call from importing `$Assumptions`. The supplied `make_certificate_patch.py` requires the exact canonical module blob, requires one matching anchor, and writes a separate candidate, unified diff, and manifest. It never overwrites the input.

This candidate has *not* been successfully tested end-to-end in the native kernel here. The isolated **Refine** behavior was observed, and the Python source-editing mechanics passed fixture tests. Those facts are weaker than a patched certificate passing its positive and negative integration cases. The generator also does not fix ambient assumptions imported earlier by other constructors or arbitrary evaluation side effects.

The broader repair is to make the proof environment explicit at certificate entry. Only retained parameter hypotheses appropriate to the certificate may participate; source predicates must be proved on the interval. A source-symbol alpha-renaming can reduce accidental name collision, but explicit assumption isolation is still necessary: renaming alone is not a principled proof-context boundary.

5.5 A correct interval obligation

For a source domain $D(x)$ and verification interval $I = [l, r]$, the relevant premise is

$$\forall x \in I, \quad D(x). \tag{3}$$

For an affine inequality $ax + b > 0$ with exact rational coefficients, its range is obtained at the endpoints. In this case the range of $x - 2$ on I is $[-3/2, -1/2]$, immediately refuting (3). No symbolic assumption solver is required.

A useful repair therefore also reorders the helper: evaluate exact rational Boolean and affine cases first; use isolated symbolic normalization only where identities such as $\text{Im } x = 0$ need it. This is a targeted

performance opportunity because it avoids unnecessary symbolic work on the newly critical path. No timing improvement is claimed without measurement.

5.6 Acceptance criteria

The contaminated and clean calls for target 1 must both fail to certify the constrained root. A valid control with target 3, interval $[5/2, 7/2]$, and center 3 should still be certifiable. Adding an unrelated ambient source assumption must not turn an invalid interval into a valid one, nor change a valid rational proof into an unrelated branch conclusion. The valid target-3 control is supplied as a regression specification and checked mathematically, but was not successfully executed through the native connector during this audit.

6 N02: composition promotes a fixed-parameter bound to a false uniform bound

6.1 The observed public result

```
Clear[x, a];
outer = AsymptoticInverse[AsymptoticExpansion[
  x/(a + x), {x, 0, 2}, Assumptions -> a > 0];
inner = AsymptoticInverse[AsymptoticExpansion[
  a, {a, 0, 4}];
r = AsymptoticInverse[SeriesCompose[outer, inner];
```

The outer object contains $x/a + \mathcal{O}(x^2)$ for fixed $a > 0$. The inner object is exactly a . The observed composition is

$$1 + \mathcal{O}(a^2), \quad a \rightarrow 0^+, \quad (4)$$

with `RemainderScaleExpression -> a^2`. However, direct substitution into the original function gives

$$\frac{a}{a + a} = \frac{1}{2}. \quad (5)$$

The error is the nonzero constant $1/2$. This is a genuinely false asymptotic statement, not a conservative or suboptimal bound.

All quantities are real on the recorded positive domains. The inner variable approaches the correct positive source endpoint. Additional sign checks or a stronger proof of $a > 0$ cannot fix this example. What changed is that a , formerly a fixed parameter of the outer germ, became the variable of the new asymptotic approach.

6.2 The missing quantifier

For fixed $a > 0$,

$$\frac{x}{a + x} - \frac{x}{a} = -\frac{x^2}{a(a + x)}. \quad (6)$$

As $x \rightarrow 0^+$ this is indeed $\mathcal{O}_a(x^2)$. The subscript matters. The assertion has the form

$$\forall a > 0 \exists C(a), \delta(a) > 0 \forall x \in (0, \delta(a)) : |R(x, a)| \leq C(a)x^2. \quad (7)$$

It does not provide constants uniform as $a \rightarrow 0$. On the diagonal $x = a$, the effective ratio $|R|/x^2 = 1/(2a^2)$ diverges. A condition that parameters are positive does not imply that constants depending on them remain bounded.

Proposition 6.1. *For $f(x, a) = x/(a + x)$ and $a > 0$, substitution of $x = a$ into the finite approximation x/a yields a constant error $1/2$. For any integer $m > 1$, substitution of $x = a^m$ yields an error*

$$E_m(a) = \frac{a^{2m-2}}{1 + a^{m-1}}, \quad (8)$$

which is not $\mathcal{O}(a^{2m})$ as $a \rightarrow 0^+$.

Proof. The diagonal identity is immediate. For $x = a^m$ the exact value is $a^{m-1}/(1 + a^{m-1})$, whereas the retained approximation is a^{m-1} . Their difference is (8). Dividing by a^{2m} gives $1/[a^2(1 + a^{m-1})] \rightarrow \infty$. $\square$

Only the $m = 1$ public composition was successfully evaluated natively. The $m > 1$ family is an independently proved extension with exact-rational Python checks, not an additional claimed native observation. It is informative because $x/a \rightarrow 0$ when $m > 1$: even moving into the genuine small-ratio regime does not justify the overstrong transported error.

6.3 Where the representation loses the dependency

The generic `SeriesCompose` computes the outer local coordinate on the inner jet, finds its valuation α , composes retained coefficient rows, and transports the outer error pair (p, d) to $(\alpha p, d)$. The relevant operation effectively adds a pure remainder at power `alpha p`. There is no fixed-parameter dependency check at this point.[10]

Retained coefficients that depend on the new variable can be handled as expressions by the coefficient application machinery. The old remainder constant has no such expression: its possible dependence on a was not retained as an explicit quantitative factor. This creates an asymmetry. The finite part is evaluated along the new path, while the unknown error is treated as though all old parameters remained fixed.

A symbol clash is therefore a semantic event, not a harmless naming issue. Renaming the outer source variable is appropriate binder hygiene, but renaming the free parameter a would change the user's function. The correct response is to detect the dependency and either establish its admissibility or refuse the unsupported composition.

6.4 Immediate repair without a general uniform-asymptotics engine

A narrow safe rule is:

When the outer operand has an unknown remainder, reject a composition whose new expansion variable occurs as a fixed parameter of that outer operand, unless retained uniformity or quantitative error evidence authorizes the new dependence.

The dependency footprint must include the original source, stored assumptions, domain conditions, and error provenance. Scanning only the displayed finite coefficients is insufficient: a parameter may first occur in an omitted term. For instance, the finite approximation to $x^2/(a + x)$ below exponent two is zero for fixed a , yet the omitted coefficient depends singularly on a .

This rule does not require banning all compositions with shared symbols. If the outer object is exact, there is no unknown outer tail to promote. The exact outer function x/a composed with the exact inner function a legitimately gives 1 on $a > 0$. This control belongs alongside the failing partial-outer example so that a fix does not indiscriminately reject harmless exact algebra.

The first diagnostic should identify the captured parameter and the failed obligation, for example:

```
Failure["NonuniformParameterCapture", <|
  "CapturedParameters" -> {a},
  "OuterApproachVariable" -> x,
  "NewApproachVariable" -> a,
  "RequiredEvidence" -> "Uniform outer remainder or transported majorant"
|>]
```

This is a proposed diagnostic, not an existing package result.

6.5 A quantitative route that really works

From (6), positivity gives

$$|R(x, a)| \leq \frac{x^2}{a^2} \quad (x, a > 0). \quad (9)$$

Unlike the bare symbol $\mathcal{O}_a(x^2)$, this majorant can be substituted. Along $x = a^m$, it becomes a^{2m-2} , the correct error power in Proposition 6.1. Along $x = a$, it becomes 1, correctly indicating that this finite approximation has no vanishing error on that path.

An eventual implementation could retain a majorant expression together with its domain and threshold dependence. Alternatively, a source-aware operation can first substitute into the exact retained source and then construct a fresh expansion of the resulting function. That yields 1/2 exactly in the observed example. Such a route should explicitly say it recomputed from the source; it is not a consequence of the old finite jet alone.

The distinction prevents another UX failure: an object with retained source data must not make operations appear to gain precision from nowhere. “Compose these approximations” and “compose the exact sources and re-expand” are different operations with different cost and information requirements.

7 N03: a truthful finite jet does not establish an analytic remainder

7.1 A real opaque function with a correct derivative

Define a real function near zero by

$$g(0) = 0, \quad g(x) = x^2 \log(1 + |\log |x||) \quad (x \neq 0). \quad (10)$$

The native reproducer deliberately supplies its numeric definition and its correct first derivative at zero:

```
Clear[g, x];
g[t_?NumericQ] := If[TrueQ[t == 0], 0,
  t^2 Log[1 + Abs[Log[Abs[t]]]]];
g'[0] = 0;

s = AsymptoticInverse[AsymptoticExpansion[g[x], {x, 0, 2}];
```

The observed result has finite expression zero and

```
"Remainder" -> PowerLogRemainder[x, 2, 0]
"Exact" -> False
"Assumptions" -> True
```

The coefficient data are not fraudulent or inconsistent. In particular,

$$g'(0) = \lim_{h \rightarrow 0} \frac{g(h) - g(0)}{h} = \lim_{h \rightarrow 0} h \log(1 + |\log |h||) = 0. \quad (11)$$

The function is continuously differentiable near zero. It is the stronger second-order magnitude remainder that fails.

7.2 Exact contradiction with the advertised bound

For $0 < x < 1$, write $L = -\log x$. Then

$$\frac{g(x)}{x^2} = \log(1 + L) \longrightarrow \infty. \quad (12)$$

Hence $g \notin \mathcal{O}(x^2)$. The native evaluator also returned `Infinity` for the equivalent scaled limit `Limit[Log[1-Log[x]],x->0,Direction->"FromAbove"]`.

The invalid remainder can look deceptively plausible numerically because its missing factor grows very slowly. At $x = e^{-k}$, the error ratio is only $\log(1 + k)$. Sampling a few small values is much weaker than this exact limit. The Python checks use the scaled expression directly to avoid underflow; they illustrate the witness but are not the proof of its limiting behavior.

The example is stronger than supplying an arbitrary unsupported symbolic head. It is a real function with an executable numeric implementation and correct finite derivative data, and the package returns a successful magnitude-bearing object. A caller should not have to infer an unstated analyticity premise from the implementation's fallback strategy.

7.3 How a formal default crosses the boundary

In the ordinary core, `fwdAnalytic` asks native `Series` for a local expansion of a unary function, while `fwdSeries` is a general fallback. The ordinary observable path also uses a native Taylor expansion for regular unary expressions. These inspected calls do not themselves turn off the default analytic treatment of an unrecognized head.[9, 10]

Wolfram documents `Analytic -> True` as the default controlling whether unknown functions are treated as analytic. This is useful for formal symbolic work. It is insufficient evidence for the concrete magnitude interpretation of `PowerLogRemainder` when the user's function is not analytic.[22]

The architecture already distinguishes some stronger cases: specialized native-function handling has its own guarded parsing and real-domain machinery. The finding is not that every adapter ignores regularity. It is that the generic fallback can admit a weaker formal result into an object whose downstream operations treat the remainder as an actual bound.[20, 21]

7.4 Repair: make regularity admission explicit

The strict fallback should not infer analyticity merely from an unknown function's ability to produce a symbolic `SeriesData`. A first hardening candidate is to request `Analytic -> False` at generic import

points and decline results whose remaining form cannot be admitted under the supported analytic model. This change must be tested with the known-function cases rather than applied as an unexamined global option mutation.

A deliberate formal mode can remain useful. It should retain something like

```
"RegularityEvidence" -> <|
  "Origin" -> "UserDeclaredAnalyticGerm",
  "Function" -> g,
  "Point" -> 0,
  "Scope" -> "Local neighborhood"
|>
```

The spelling is a design proposal. The essential requirement is that a formal premise remain visible and that certificate consumers not mistake a user assertion for an independently proved analytic fact.

Analyticity is sufficient but not always necessary. A bounded second derivative on a neighborhood would justify a second-order Taylor magnitude remainder in this example's role. More generally, the admission layer should accept the particular differentiability and derivative-bound evidence an operation needs. A finite table of derivatives at one point is not such neighborhood evidence.

No native integration result is claimed for these repairs. Attempts to evaluate strict-mode and patched controls encountered connector failures. The supplied desired-contract test permits either rejection or a genuinely sufficient conservative remainder; it does not prescribe one exact implementation response.

7.5 Consequences for tests and API explanations

The regression should retain the truthful derivative definition. Replacing the example with a deliberately false derivative would test user inconsistency, not the package's regularity inference. A known analytic function such as e^x must remain a positive control, and a returned unknown-function expansion must say which regularity premise was used.

A diagnostic such as "Unable to establish a magnitude remainder for this opaque function" is more informative than "Unsupported input." It tells the caller that the missing item is not a coefficient but an admissibility condition. The package can still provide a formal jet as a different kind of result, provided its consumers respect the distinction.

8 N04: codomain admission is inconsistent across public routes

8.1 Observed acceptance and rejection controls

Under $a > 0$, the principal value $\sqrt{-a} = i\sqrt{a}$ is provably nonreal. The following direct expansion nevertheless succeeds:

```
AsymptoticInverse'AsymptoticExpansion[
  Sqrt[-a] + x, {x, 0, 2}, Assumptions -> a > 0]
```

Its expression is $\sqrt{-a} + x$, its remainder is zero, and its recorded target domain is $x > 0$. The same successful result is obtained by applying an additive observable:

```
base = AsymptoticInverse'AsymptoticExpansion[
  x, {x, 0, 3}, Assumptions -> a > 0];
AsymptoticInverse'SeriesObservable[base, z + Sqrt[-a], z]
```

By contrast, the native controls

```
AsymptoticInverse[SeriesAdd[base, Sqrt[-a]]
AsymptoticInverse[AsymptoticInverse[
  x + Sqrt[-a] x^2, {x, 0}, {y, 3}, Assumptions -> a > 0]
```

correctly return `Failure["UnprovedRealCoefficient",...]`.

The inverse control is important. It would be inaccurate to claim that the ordinary inverse constructor generally lacks the real-coefficient guard. The demonstrated gap is in direct/independent coefficient admission and its relation to other public routes.

8.2 Why syntactic exactness is not a realness proof

A symbolic expression can be exact and contain no literal `Complex` object while still being nonreal under the supplied assumptions. Testing for machine numbers or explicit complex atoms does not prove $c \in \mathbb{R}$.

The ordinary constant representation `pConst` checks the logarithmic polynomial shape and zero status, but does not establish realness on its own. The forward dispatcher and independent-coefficient application routes can reach it directly. Other public scalar routes separately prove real coefficients and reject this example.[9, 10]

The result $x + i\sqrt{a}$ is a perfectly meaningful exact complex expression. What is missing is a coherent contract saying that this route intentionally constructs a complex-valued germ, with compatible downstream operations. The inspected package describes real-branch computation, and its existing scalar/inverse guards make the accidental discrepancy visible.[1, 9, 10]

8.3 Repair at the correct abstraction level

The shared low-level coefficient machinery should not simply be modified to reject every complex intermediate. Fourier representations naturally use complex modes whose conjugacy yields a real function, and other native transformations may temporarily introduce complex algebra.[14, 20]

Instead, every public real-valued constructor or operation should pass a common admission boundary after its representation is normalized. That boundary needs three outcomes: proved real, proved nonreal, and unresolved under the stated assumptions. The last two may both decline in a strict real-only mode, but diagnostics should distinguish them when possible.

A reusable coefficient check should be keyed by the normalized coefficient, the explicit assumptions, the coordinate domain, and the requested codomain. It should be evaluated once per distinct coefficient within an operation, not repeatedly in every sparse product. This both closes the route discrepancy and avoids turning validation into a large new symbolic cost.

The API-level invariant is straightforward: explicit `SeriesAdd[s,c]` and the mathematically equivalent additive observable must not disagree on whether a provably nonreal constant is allowed. A future complex-valued mode could make both succeed, but that would require explicit codomain and branch/error rules. Merely allowing one path to slip through is not that feature.

9 A repair architecture focused on the demonstrated defects

9.1 Separate four kinds of context

The examples reveal four independent items that should not be merged into one undifferentiated assumption expression:

$$\mathcal{C} = (\Gamma_{\text{param}}, D_{\text{source}}, A_{\text{approach}}, H_{\text{regularity}}). \quad (13)$$

Here Γ_{param} describes fixed parameters, D_{source} is the source domain to be checked, A_{approach} describes the limiting variable and side, and $H_{\text{regularity}}$ describes analytic or derivative-bound evidence. A codomain judgment and a fixed-parameter footprint accompany the representation.

This need not require changing the public wrapper head. The association payload can gain explicit evidence records, while existing coefficient engines receive the same normalized jets. The important change is that operation admission examines these records before interpreting successful algebra as a successful analytic result.

For a supported function F , a useful internal judgment is

$$(\mathcal{C}, \mathcal{P}) \vdash F = J + R, \quad |R| \preceq B, \quad (14)$$

where $\mathcal{P}$ identifies fixed parameters and B is either a qualitative scale with stated dependence or a quantitative majorant. The symbol $\preceq$ must say whether constants and thresholds may depend on $\mathcal{P}$.

N01 improperly uses D_{source} as though it were an ambient premise proving itself on an interval. N02 changes $\mathcal{P}$ without revalidating B . N03 creates (14) from a formal coefficient model without $H_{\text{regularity}}$. N04 omits a codomain premise on some public routes. This classification gives repair locations more precise than “validate everything.”

9.2 An implementable sequence, not a monolithic rewrite

First isolate the certificate domain normalization and add the invalid-interval regression. Next add a conservative dependency check to partial-outer composition. Both changes can be local and fail closed on exactly the newly identified unsupported cases.

Then add generic regularity admission at native fallback boundaries, while retaining explicit known-family paths. Finally centralize public codomain admission and route-equivalence tests. These changes can be reviewed independently; they should not be hidden inside an unrelated optimization or a new special-function family.

Only after those guards are stable is it necessary to add richer positive evidence: transported parameter-dependent majorants, user-declared analytic germs, and source-backed re-expansion. This ordering produces correct refusals immediately while keeping the eventual richer behavior grounded in a clear obligation.

9.3 Proof caching must retain its dependencies

Several proposed repairs invite caching, but a cache keyed only by a coefficient or predicate is unsafe. The same expression can be real under one parameter domain and nonreal under another; a uniformity statement can hold on a compact parameter set and fail near a singular boundary. An ambient assumption must never become a hidden cache dependency.

A per-request cache is a practical first implementation. Its key contains normalized expressions and an immutable explicit context identifier. Public operations construct the identifier from recorded evidence,

not from a dynamically consulted global assumption variable. Cross-request reuse can be considered later only when the evidence and parameter bindings are stable.

For N01, no cached Boolean claim that $x > 2$ is true may stand in for evaluation on a different interval. For N02, a cached coefficient expansion at fixed a cannot authorize the diagonal path. The same discipline that prevents these errors also makes profiling interpretable: cache hits then mean reuse of the same mathematical obligation.

9.4 Do not solve a metadata problem by silently recomputing everything

The exact source retained by an object is valuable, but repeatedly invoking it can hide a broken approximation contract. In N02, recomputing the exact diagonal source gives the right answer. In a general object, the source may be expensive, unavailable, a user declaration, or only a truncated model. A source-aware operation must disclose its origin and preserve any uncertainty in that source.

Consequently a result should distinguish coefficient algebra performed on the existing approximation from reconstruction using stronger source information. The latter may legitimately improve precision, but that improvement must not be attributed to the former. This is a targeted extension of provenance semantics required by the proposed N02 repair, not a new report of the already catalogued provenance-storage issue.

10 Performance and UX implications specific to these findings

10.1 Four targeted measurements worth adding

No new wall-clock measurements were obtained, so the following are proposed measurements rather than claimed performance defects or speedups.

For the certificate helper, count calls to symbolic `Refine` separately from affine interval tests before and after reordering. A rational interval predicate should take the deterministic path; proof-time failures should report which normalization remained necessary.

For composition, measure the parameter-footprint check before coefficient composition. A stored footprint should make a rejected diagonal case inexpensive. Exact-source replay should be measured separately because it is a different operation, potentially far more costly than transporting a jet.

For regularity admission, count native fallback attempts for opaque functions under strict and declared-analytic modes. A refusal should not repeatedly call the same native expansion at increasing orders when the missing premise is regularity rather than truncation depth.

For realness admission, count distinct normalized coefficients and proof attempts. The useful ratio is proofs per distinct coefficient under the same context, not proofs per generated product row. This identifies whether the new guard is being placed too deep in the arithmetic loop.

These measurements attach directly to N01–N04. They do not reissue the existing register’s general term-budget, dense-export, or provenance-growth findings.[3]

10.2 Error messages should state the lost premise

N01 should report an interval/domain failure with the retained predicate and verification interval. N02 should identify the captured fixed parameter and the need for a uniform bound or source re-expansion. N03 should distinguish an unavailable analytic/magnitude contract from unavailable coefficients. N04

should report the coefficient and the real-domain obligation that failed.

A successful result should be equally explicit. A user-declared analytic model should not look indistinguishable from a known-family theorem. A source-recomputed composition should say that it used stronger source information. A certificate should state its actual proof assumptions and scope, not merely a global success flag. These are narrowly targeted explanations of the demonstrated gaps.

10.3 A specific safe-but-unsupported observable opportunity

The current Gamma/Barnes inverse power route rejects a noninteger outer power whenever the source endpoint is not positive infinity. This is conservative: a negative source branch can create principal-power hazards. The implementation correctly avoids inventing a branch.[18]

There is nevertheless a useful admissible subcase. Suppose $X(y) < 0$ is the selected source root and the existing observable is $X(y)^2$. Then its positive square root is

$$\sqrt{X(y)^2} = -X(y), \quad (15)$$

not $X(y)$. A safe extension would track the sign of the *observable*, separately from the sign of the source root, and reconstruct a signed source-power observable. Merely deleting the guard and multiplying exponents would be wrong.

For an even integer source power m , a real outer power k , and a negative real source root, the principal real value is $|X|^{mk}$. When mk is an integer it can be represented as $(-1)^{mk} X^{mk}$; otherwise an explicit positive-magnitude carrier is needed. This is a mathematical design opportunity, not an observed incorrect current result, and no native acceptance test was completed for it. The supplied independent checks cover only the elementary sign identity.

11 Acceptance plan and reproducibility

11.1 Tests should check logical strength, not just finite coefficients

The desired-contract suite contains twelve cases. It includes the contaminated and clean certificate exclusion cases; a valid constrained-root control; partial-outer diagonal rejection or exact source re-expansion; an exact-outer composition control; the opaque-function witness and a known-analytic control; and direct, observable, scalar, inverse, and real-parameter codomain controls.

The diagonal test intentionally distinguishes the chosen conservative repair from an explicit exact-source route. A refusal is acceptable for the former; an exact value 1/2 is acceptable for the latter. A displayed value 1 with a small remainder is not. The opaque-function test allows rejection or a sufficient error bound rather than demanding one error tag.

The suite is stricter than checking that no message was emitted or that an expression has the expected head. The relevant assertions are whether the domain is actually satisfied, whether the error divided by its claimed scale remains bounded, and whether equivalent public routes agree about codomain admission.

11.2 What was executed and what was not

Activity	Status	Interpretation
Four main native reproductions	Executed	Actual pinned-package behavior.
N01 clean exclusion and N04 scalar/inverse controls	Executed	Important negative controls, not inferred outputs.
Isolated native Refine primitive comparison	Executed	Confirms the ambient-context mechanism.
Ten Python mathematical checks	Passed	Exact witnesses and limited numeric illustrations; not package execution.
Five Python patch-mechanics checks	Passed	Small explicit fixtures only.
Twelve packaged Wolfram regression tests as a suite	Not run	Desired behavior, including expected baseline failures.
Patched certificate integration	Not completed	Service failures prevented a successful repair run.
Full upstream native suite	Not run	No full-suite or release-acceptance claim.
Optional built-in comparison harness	Not run	Reference-based comparison in the article, reproducible probes supplied.

11.3 Using the artifacts

`code/reproduce.wl` records observations rather than claiming baseline bugs are passing tests. Its shared loader downloads the exact commit by default and verifies the standalone Git blob before loading. Set `ASYMPTOTIC_AUDIT_SOURCE` to use a locally downloaded standalone. A deliberate `ASYMPTOTIC_AUDIT_ALLOW_MODIFIED=1` override permits a reviewed rebuilt candidate; the output records that override. The override is not a recommendation to load arbitrary code.

```
python code/independent_checks.py
python -m unittest discover -s code -p test_patch_generator.py -v
wolframscript -file code/reproduce.wl
wolframscript -file code/run_regressions.wl
wolframscript -file code/compare_builtins.wl
```

The packaged Wolfram wrappers themselves were not executed as local command-line programs here. Equivalent expressions for the observations were evaluated through the connector. The `native-observations.json` file explicitly identifies its contents as transcribed connector evidence, not an automated capture of a wrapper run.

To generate the certificate candidate from the canonical module in an exact checkout:

```
python code/make_certificate_patch.py \
/path/to/Asymptotic/AsymptoticInverse/Kernel/InverseCertificates.wl \
--out candidate-certificate
```

Review the resulting diff in a separate worktree, apply it to the canonical module, regenerate the root standalone using the upstream builder, and run the focused tests before broader validation. The generator checks source identity after LF normalization and refuses mismatches, duplicate anchors, and an existing

output directory.

11.4 Recommended repair order

N01 is first because it invalidates an explicit positive certificate. N02 follows because it returns a false asymptotic approximation through a normal public operation. N03 then closes the formal-to-magnitude boundary. N04 centralizes public codomain admission without banning valid internal complex representations. The optional sign-aware Gamma/Barnes observable extension belongs after these repairs, not before them.

Each repair should be reviewed against its positive control so that a safety guard does not become an indiscriminate rejection of supported mathematics. The proof obligations are small enough to be stated precisely and tested independently. That is the most direct path from the current implementation to a more dependable composition of its already substantial capabilities.

A Source coverage and limits

The source review emphasized the ordinary forward/operation paths and their interfaces with specialized families. The inventory contains 38 kernel `.wl` modules plus `init.m`; this report does not claim a line-by-line review of all of them. The machine-readable `evidence/coverage.json` records inspected ranges and explicitly excluded coverage claims.[8]

The ordinary canonical module was read through its forward engine; `SeriesOperations.wl` was read across overlapping ranges through its final refinement dispatch. Substantial certificate ranges included the entire source-domain predicate helper. Complete or extensive reads also covered Gamma/Barnes forward and inverse code, native special-function adaptation, exact identities, parameterized special functions, flat-sector construction/operations, Fourier coefficients, reciprocal-logarithmic operations, source charts, Dirichlet expansions, and numerical inverse checks. Some long connector responses were truncated near their end; the coverage manifest records that limitation instead of treating a requested range as automatically fully read.

The mathematical article in the repository, all notebooks, every test file, and all specialized proof claims were not independently reverified. In particular, the report does not infer full correctness of a specialized family merely because its inspected recurrence looked consistent. It records positive architectural observations only where supported by the inspected source.

B Exact witnesses and useful variants

B.1 Certificate witness without asymptotic numerics

Let $D = (2, \infty)$, $I = [1/2, 3/2]$, and $F(x) = x - 1$. Then $F(1) = 0$, $1 \in I$, $I \cap D = \emptyset$. Any output asserting both a root enclosure inside I and verification of the source predicate throughout I is inconsistent with these exact rational facts. No floating-point accuracy, conditioning, inverse approximation quality, or difficult transcendental inequality is involved.

B.2 A family of diagonal failures

For $c > 0$, substitution $x = ca$ into $x/(a + x)$ gives $c/(1 + c)$, whereas substitution into x/a gives c . The error $c^2/(1 + c)$ is independent of a . This family shows that the observed $1/2$ discrepancy is not a special

cancellation at one parameter value.

If the outer expansion retains N geometric terms,

$$\frac{x}{a+x} = \sum_{j=1}^N (-1)^{j-1} (x/a)^j + \frac{(-1)^N (x/a)^{N+1}}{1+x/a}, \quad (16)$$

then along $x = ca$ the error remains a nonzero constant for each fixed finite N . Increasing the ordinary source cutoff without changing the parameter contract does not solve the issue. Along $x = a^m$, the actual error power is $(m-1)(N+1)$ rather than the naively transported $m(N+1)$. These are mathematical extensions, not additional native measurements.

B.3 The opaque function’s derivative really is continuous

On $0 < x < 1$,

$$g'(x) = 2x \log(1 - \log x) - \frac{x}{1 - \log x}. \quad (17)$$

Both terms tend to zero. The definition is even, so the negative-side derivative tends to zero as well. Thus the supplied derivative value at zero is compatible with a C^1 function. Its second derivative is unbounded near zero, consistent with the failure of an $\mathcal{O}(x^2)$ first-order Taylor remainder.

The witness separates three statements: the derivative at zero exists; the first-order Taylor remainder is $o(x)$; and that remainder is $\mathcal{O}(x^2)$. The first two hold, while the third does not. A successful finite Taylor-coefficient computation alone does not bridge that gap.

C Artifact manifest and build instructions

The archive contains the article source and PDF, a pinned native loader, observation and comparison harnesses, twelve desired-contract tests with a runner, the certificate-patch generator, independent mathematical checks, patch-mechanics tests, evidence manifests, test logs, and checksums. It does not include the upstream package, a complete upstream checkout, or font files.

Build the article with a standard TeX Live installation containing Latin Modern, AMS packages, microtype, listings, booktabs, longtable, fancyhdr, xurl, and hyperref:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error asymptotic-contract-audit.tex
pdflatex -interaction=nonstopmode -halt-on-error asymptotic-contract-audit.tex
```

The delivered PDF was compiled and rendered for layout inspection. Build timestamps or TeX-version differences can change the PDF byte hash on rebuilding; the source and evidence content are the reproducibility reference. The `SHA256SUMS.txt` file identifies shipped artifacts, while Git blob identifiers identify the reviewed upstream source. These are different uses of hashes.

References

- [1] Vladimir Reshetnikov, Asymptotic repository, [README](#), pinned revision 921387e5ba12. All repository references below use this same full revision unless explicitly noted.
- [2] Asymptotic, existing code-review index.

- [3] Asymptotic, consolidated code-review status register.
- [4] Asymptotic, review 4 article, inspected opening assessment and evidence scope.
- [5] Asymptotic, review 8 README, historical native evidence and limited patch scope.
- [6] Asymptotic, generated standalone package, provenance header and pinned artifact identity.
- [7] Asymptotic, standalone builder.
- [8] Asymptotic, pinned kernel directory inventory.
- [9] Asymptotic, canonical ordinary kernel; especially `pConst`, `fwd`, `fwdAnalytic`, `fwdSeries`, and precision-tracked jet helpers.
- [10] Asymptotic, series operations; especially `SeriesCompose`, `seriesIndependentJet`, `seriesJetApply`, `SeriesObservable`, and `seriesConstant`.
- [11] Asymptotic, inverse certificates; especially `certPositiveCondition`, source-domain extraction, and certificate dispatch.
- [12] Asymptotic, flat-sector inverse construction.
- [13] Asymptotic, flat-sector operations.
- [14] Asymptotic, Fourier-polynomial coefficient engine.
- [15] Asymptotic, reciprocal-logarithmic calculus.
- [16] Asymptotic, Gamma inverse construction.
- [17] Asymptotic, Barnes inverse coefficients and branch evidence.
- [18] Asymptotic, Gamma/Barnes inverse observable powers.
- [19] Asymptotic, exact special-function identities.
- [20] Asymptotic, native special-function adaptation.
- [21] Asymptotic, special-function real-domain contracts.
- [22] Wolfram Research, *Series*, official Wolfram Language documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/Series.html>.
- [23] Wolfram Research, *SeriesData*, official documentation. <https://reference.wolfram.com/language/ref/SeriesData.html>.
- [24] Wolfram Research, *InverseSeries*, official documentation. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [25] Wolfram Research, *ComposeSeries*, official documentation. <https://reference.wolfram.com/language/ref/ComposeSeries.html>.
- [26] Wolfram Research, *Asymptotic*, official documentation. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [27] Wolfram Research, *AsymptoticSolve*, official documentation. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [28] Wolfram Research, *Refine*, official documentation, especially positional and option assumptions. <https://reference.wolfram.com/language/ref/Refine.html>.